Regulatory & Immutable Records + File Plan — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 4

1. Solution Overview — Regulatory & Immutable Records + File Plan (Microsoft Purview) 4

What this solution is . . . 5

The core distinction — retention vs a record vs a regulatory record . . . 5

The mapped file plan . . . 5

What this solution is NOT (the deferred boundary) . . . 6

Why this matters . . . 6

How it works . . . 7

The reference design . . . 7

Naming convention (used in every doc and script) . . . 8

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 9

2. Requirements 9

2.1 Compliance licensing for records management & the file plan . . . 9

2.2 The records determination (input from records & legal) . . . 10

2.3 Roles for records management, the file plan & disposition . . . 10

2.4 In-scope workloads and their state (Exchange / SPO / OneDrive / Teams) . . . 10

2.5 Existing Purview records posture (alignment, not duplication) . . . 11

2.6 SharePoint records mechanics (referenced, not rebuilt) . . . 11

2.7 Records/file-plan automation toolchain . . . 12

2.8 Change control and a documented rollback — with the irreversible parts flagged . . . 12

Global Requirements — Microsoft Purview Solutions 12

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 13

2. Purview roles (least-privilege) and portal access . . . 13

3. Security & Compliance PowerShell + Graph toolchain . . . 13

4. Data sources & scope in scope . . . 14

5. Compliance, records, privacy & legal stakeholders . . . 14

6. A pilot scope and change control . . . 14

3. Implementation Guide 15

Stage 0 — Connect and confirm prerequisites . . . 15

Stage 1 — Assess (read-only) . . . 15

Stage 2 — Design (map the determination to Purview) . . . 16

Stage 3 — Configure (build, staged safe) . . . 16

Stage 4 — Pilot (prove on the bounded scope) . . . 17

Stage 5 — Validate . . . 18

Stage 6 — Expand (enforce to production scope) . . . 18
Rollback (per stage) . . . 18
5. Testing & Validation 19
6. Troubleshooting 21
7. Best Practices 24
Governance & ownership . . . 24
Immutability discipline — record vs regulatory record . . . 24
The file plan — map everything, from day one . . . 25
Change-safety . . . 25
Operations & evidence . . . 25
Alignment with the broader program . . . 26
8. Deliverables Summary 26
Documentation . . . 26
PowerShell scripts (5) . . . 27
Appendix A — PowerShell Scripts 29

Executive Summary

Some content an organization holds must be more than *retained* — it must be **immutable**: locked against edit, deletion, and early disposal for its full obligation, and **mapped** to the authority that requires it. When that is left unaddressed, it is both a **compliance gap** (the record is not actually protected) and a **data-risk exposure** (there is no defensible inventory of what is locked and why). This solution delivers exactly that layer in **Microsoft Purview Records Management**: **record labels** that lock content in place, **regulatory-record labels** for the one-way cases regulators demand, and a **mapped file plan** — every record tagged with its retention authority, category, department, and citation — configured, piloted on a bounded scope, validated, and made operable, so the immutability capability is in place and defensible.

Outcome. Immutable records enforced by the platform: a **record** label declares an in-place record and locks it against edit and early deletion (reversible only by a records manager, evidenced); a **regulatory-record** label is the irreversible, cannot-be-shortened lock for genuine legal mandates; and a **mapped file plan** turns the whole records inventory into evidence — every locked record traces to an authority and a citation, exportable on demand. Records that were "kept somewhere, we think" become a reportable, defensible, tamper-proof inventory.

Approach. The scope is deliberately the **locking + mapping layer**, and it is **aligned to the organization's broader Microsoft Purview program** — it reuses that program's foundations rather than rebuilding tenant-wide retention policies, event-based-retention programs, the full disposition-review operation, DLP, or sensitivity labels, and it defers SharePoint library-level mechanics to the SharePoint records product. It **implements the records determination provided by records, compliance, and legal**; it does not author lock periods, decide record status, or make the regulatory-applicability call. Delivery follows a disciplined, pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with file-plan descriptors built first, record labels created and mapped, regulatory labels gated behind explicit sign-off and proven only on a disposable test item, validated against explicit pass/fail gates, and only then expanded to production. Five PowerShell tools (Security & Compliance PowerShell and Graph) handle scope/health, posture assessment, file-plan configuration, record/regulatory-label configuration, and compliance reporting — read-only or dry-run (`-WhatIf`) by default, with no credentials in code.

Why it is low-risk. Records declaration — and especially the **regulatory** rung — is high-impact and partly irreversible, so nothing touches production content until it is proven on a pilot. Assessment is read-only; record labels are published to the pilot scope; a regulatory label is applied only to a single disposable test item before any expansion; every record label must be **mapped** to an authority and citation or the tooling flags it; and every change has a documented rollback with the **irreversible** actions (regulatory records, completed disposition) explicitly flagged no-rollback and gated on records/ legal sign-off. Plan- and cloud-dependent behavior (records management, the file plan, auto-apply, Government-cloud portal paths) is flagged for tenant verification rather than assumed. The result: records that must be immutable are genuinely locked and fully mapped — a compliance gap and a data-risk exposure turned into a defensible, evidenced records control.

1. Solution Overview — Regulatory & Immutable Records + File Plan (Microsoft Purview)

What this solution is

Some content an organization holds is not merely *retained* — it must be made **immutable**: locked against edit, deletion, and early disposal for its full obligation, and **mapped** to the authority that requires it. This solution is a repeatable, pilot-first method to deliver exactly that layer in **Microsoft Purview Records Management**: **record labels** that lock content in place (immutable records), **regulatory-record labels** for the one-way, cannot-be-shortened cases regulators demand, and a **mapped file plan** — every record label tagged with its **retention authority, category, department, citation, and reference ID** so the records inventory is reportable and defensible. It is configured, **piloted on a bounded scope**, validated against explicit pass/fail gates, and made operable — so the immutability capability is in place and defensible, not just described in a binder.

The scope is deliberately **the locking + mapping layer**, not the whole retention program. Microsoft Purview separates *data lifecycle management* (retention policies and labels as the tenant-wide floor) from *records management* (records declaration, regulatory records, the file plan, disposition). This product implements the **records-immutability and file-plan** portion in depth. Where tenant-wide retention policies, event-based retention as a program, or a full disposition-review operation are needed, the broader **data lifecycle & records management** product covers them — this solution **references** that layer rather than rebuilding it, and keeps label names consistent with it.

The core distinction — retention vs a record vs a regulatory record

The single most important design decision here is *how strong* the lock must be, and this solution treats it as first-class:

- A **retention label** keeps (and/or deletes) an item for a duration, but the item stays **editable**

until disposal. It is retention, not immutability.

- A **record label** (`-IsRecordLabel $true`) **declares the item a record**: it becomes an in-place

record — it stays where it lives but is **locked against edit, deletion, and move** per its version and rules, with an auditable declare/undeclare trail. A records manager can **un-declare** a non-regulatory record (evidenced) if the schedule allows.

- A **regulatory-record label** (`-Regulatory $true`) is the strongest lock: it is **one-way**. Once

applied it **cannot be removed, changed, un-declared, or shortened** — not by an admin, not by a records manager. Use it **only** where records/legal explicitly require an irreversible lock.

Immutability is a spectrum from "retained but editable" to "locked but reversible" to "locked forever." This solution configures the **record** and **regulatory-record** rungs and maps them to the file plan; which rung each category needs is a **records/legal determination** it takes as input.

The mapped file plan

A locked record is only defensible if you can say **why** it is locked and **under what authority**. The **file plan** in Microsoft Purview attaches descriptors to each retention/record label:

- **Retention authority** — the schedule or mandate that requires the record (e.g. a records schedule).
- **Category / sub-category** — the records taxonomy the item belongs to.

- **Department** — the owning business function.
- **Citation** — the specific regulatory/legal reference (with an optional URL).
- **Reference ID** — the stable identifier the records program uses.

With these populated, the file plan is a reportable inventory: every immutable record maps to an authority and a citation, labels can be **exported/imported in bulk**, and an auditor's "show me your records inventory and what requires each" is a report, not a fire drill.

What this solution is NOT (the deferred boundary)

- **The records determination is not made here.** *Which* content is a record, *which* rung it needs

(record vs regulatory record), *how long* it is locked, the **retention authority/citation**, and the **disposition authority** are determined by the organization's **records-management, compliance, privacy, and legal** functions. This solution **implements** those determinations faithfully; it does not author retention periods, decide record status, or make regulatory-applicability calls.

- **Tenant-wide retention policies, event-based-retention programs, and full disposition-review

operations **belong to the broader** data lifecycle & records management** product — referenced and aligned to, not rebuilt here.

- **Legal holds and eDiscovery determinations** (what to preserve for a matter) belong to legal/

eDiscovery, not to this records configuration.

- **Sensitivity (information-protection) labels and DLP** are the province of the broader Purview

program — aligned to, not re-implemented.

- **SharePoint library-level records mechanics** (default library labels, in-place records inside

libraries, PnP-driven application) are the SharePoint records/retention product's province — referenced, with label names kept consistent.

Why this matters

- **A retention schedule that isn't enforced is not a control.** A **record** label in Microsoft

Purview is enforcement the platform executes — the item is genuinely locked against edit and early deletion, consistently, without relying on users to behave.

- **Immutability is the defensibility a regulator expects.** For records under a mandate, "we intended

to keep it" is not enough; a **regulatory-record** label is an irreversible lock the platform can attest to.

- **A mapped file plan turns records into evidence.** Authority, category, department, citation, and

reference ID make the whole records inventory reportable and traceable to the mandate that requires it — the difference between "we have records somewhere" and "here is our records inventory and what each is locked under."

- **Doing this pilot-first avoids the expensive failure mode.** A wrong record declaration locks live

content owners out; a mistaken **regulatory** label is **irreversible**. Proving the design on a bounded pilot with disposable test items — before any production content is touched — is the whole point.

How it works

1. **File-plan descriptors** are created first in Microsoft Purview → **Records management** → **File plan**

(or via `New-FilePlanProperty*` in Security & Compliance PowerShell): the **authority**, **category / sub-category**, **citation**, **department**, and **reference ID** values the labels will carry.

1. **Record labels** are created (`New-ComplianceTag -IsRecordLabel $true`) with a retention action and

duration and the **file-plan descriptors attached** — these lock content in place when applied.

1. **Regulatory-record labels** (`New-ComplianceTag -Regulatory $true`) are created **only** for the

categories records/legal designate as one-way, gated behind an explicit sign-off — an irreversible lock.

1. **Labels are applied** by publishing them (users apply the record label to declare a record),

auto-applying by condition, or setting a library default (SharePoint mechanic deferred). Applying a record label **declares the record** and locks the item.

1. **The file plan** is now a reportable inventory: each label maps to authority/category/department/

citation/reference ID, and labels can be exported/imported in bulk.

1. **Disposition at end of life** (for records that end in a reviewed disposal) routes to the Purview

Disposition queue where a named reviewer acts — this solution wires the reviewer role and reporting and defers the disposition *operation* to the broader records product.

1. The whole configuration runs through the house model — **Assess → Design → Configure → Pilot →**

Validate → Expand** — so immutability is proven on a bounded scope before any production record is declared.

The reference design

• **Inputs (from records/legal):** for each in-scope category — record status (record vs **regulatory

record**), lock duration and trigger, retention** authority**,** citation**,** department**,** reference ID**, and whether disposal is automatic or reviewed.

• **File plan (built first):** the descriptor set — `Authority – Records-Schedule-2026`, categories,

citations, departments, reference IDs — so every label is mapped from creation.

• **Record labels (immutable, reversible):** one record label per schedule category that needs an

in-place lock, named consistently, with file-plan descriptors attached — e.g. `Record Label – Contracts-7yr – Pilot`.

• **Regulatory-record labels (one-way):** only where records/legal require an irreversible lock —

e.g. `Regulatory Record Label – Financial-7yr – Pilot` — gated behind explicit sign-off.

• **Application: published** label policies where users declare records manually; **auto-apply**

policies for categories detectable by condition; SharePoint default-library labels deferred to the SharePoint records product.

• **Scope:** a **static** pilot scope for validation; expansion to production (adaptive scopes optional)

only after the gates pass. *Records management, the file plan, regulatory records, and auto-apply are plan-dependent — verify your licensing in your tenant.*

Naming convention (used in every doc and script)

Every artifact is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and Purview admin surfaces:

Artifact	Name
Record label (immutable, reversible)	`Record Label – Contracts-7yr – Pilot`
Regulatory-record label (one-way)	`Regulatory Record Label – Financial-7yr – Pilot`
Published label policy (manual declare)	`Label Policy – Records-Publish – Pilot`
Auto-apply label policy	`Label Policy – Records-AutoApply – Pilot`
File-plan retention authority	`Authority – Records-Schedule-2026`
File-plan category	`Category – Contracts`
File-plan citation	`Citation – SOX`
File-plan department	`Department – Finance`
File-plan reference ID	`ReferenceId – REC-CON-001`
Pilot scope (static)	`Scope – Records-Pilot – Static`
Disposition reviewer group	`Disposition Reviewers – Records – Stage1`
Assessment export	`Records Posture – Tenant – Discovery`
Compliance report	`Records & File Plan – Tenant – Report`

When the pilot passes and you expand, mint the production equivalents by swapping the `– Pilot` stage for `– Prod` (e.g. `Record Label – Contracts-7yr – Prod`) — never edit a validated pilot artifact in place, and never re-declare a regulatory record.

Limitations (be honest with buyers)

• **Records management and the file plan are plan-dependent.** Record/regulatory-record labels, the file plan, auto-apply, and adaptive scopes generally require an **E5 / E5 Compliance-class** plan or add-on — **verify what your tenant is licensed for.**

• **Regulatory records are one-way and irreversible.** A regulatory-record label cannot be removed, changed, un-declared, or shortened. Use it only where records/legal explicitly require it, and never apply it to content you are not prepared to keep for its full period.

• **A record label locks content by design.** Owners/users lose edit/delete on declared items; communicate this before applying record labels to live content.

• **Declaration and propagation are eventually consistent, not instant.** Label publish/auto-apply and declaration run on service timers (hours to several days; auto-apply of existing content can take 7+ days) — do not judge a pilot on the first minutes.

• **Disposition is a portal workflow.** Reviewers act in the Purview **Records management** → **Disposition** area; this solution wires the role and reports coverage, and points reviewers to the queue rather than approving disposals from a script.

• **Commercial vs Government cloud paths differ.** Portal URLs and some feature availability differ in GCC / GCC High / DoD (e.g. `*.office365.us`, `Connect-IPPSSession -ConnectionUri`) — **verify the portal path and feature in your cloud.**

The standard deployment model

> ***Assess*** *current records/file-plan posture and ingest the records determination →* ***Design*** *the record/regulatory-label + file-plan map →* ***Configure*** *file-plan descriptors, record labels, and (where required) regulatory labels, staged safe →* ***Pilot*** *on* `Scope – Records-Pilot – Static` *(publish/auto-apply to the pilot, declare disposable test items) →* ***Validate*** *against explicit pass/fail gates →* ***Expand*** *to production (adaptive scopes optional) under change control.*

Records declaration — and especially **regulatory** declaration — is high-impact and partly **irreversible**. Every production change goes through change control with a documented rollback that explicitly flags the no-rollback parts, and nothing expands past the pilot until the Section 5 gates pass.

2. Requirements

These are the **solution-specific** prerequisites for delivering regulatory & immutable records and a mapped file plan in Microsoft Purview. They build **on** the shared Purview category requirements in `global-requirements-purview.md` (a Microsoft 365 tenant with the required Purview/compliance licensing; least-privilege Purview roles and portal access; the Security & Compliance PowerShell + Graph toolchain; the in-scope data sources/workloads; the compliance/records/privacy/legal stakeholders; and a pilot scope with change control). Do not repeat those — verify them first, then confirm the records-immutability and file-plan-specific prerequisites below. Each is stated as **Description / Why / Verify / Implement if missing.**

2.1 Compliance licensing for records management & the file plan

• **Description:** The Microsoft 365 / Microsoft Purview licensing that unlocks **records management** — **record labels**, **regulatory-record labels**, the **file plan** and its descriptors, **auto-apply** label policies, and **adaptive scopes** — which is generally an **E5 / E5 Compliance-class** plan or the equivalent Purview add-on. (Basic retention labels are broadly available; the immutability and file-plan features this solution centers on are the plan-gated ones.)

• **Why required:** These capabilities are **plan-gated**; without the right plan the record/regulatory labels and file-plan descriptors in the design cannot be created.

• **Verify:** In the Microsoft 365 admin center → Billing → Licenses, confirm the compliance plan/add-on.

In the Microsoft Purview portal, confirm the **Records management** solution (with **File plan** and **Disposition**) is visible. *Feature-to-plan mapping changes — verify the exact entitlement in your tenant.*

• **Implement if missing:** Have licensing assign/purchase the required compliance plan or add-on to the admin and any accounts that will manage records, before configuration.

2.2 The records determination (input from records & legal)

• **Description:** The authoritative determination for the in-scope content: for each category, whether it is a **record** or a **regulatory record**, the **lock duration** and its **trigger** (created / last-modified / labeled / event), the applicable **retention authority** and **regulatory citation**, the owning **department** and **reference ID**, and the **disposition authority** (automatic vs reviewed, and who reviews).

• **Why required:** This solution **implements** the determination; it does not author it. Record status (especially the irreversible **regulatory** designation), lock periods, and citations are legal/records calls — without them there is nothing correct and defensible to configure.

• **Verify:** Confirm records/legal has provided (or signed off) a records determination naming each category's record status, lock period/trigger, authority, citation, department, reference ID, and disposition path — or at least a pilot subset of categories, and an explicit sign-off for any **regulatory-record** category.

• **Implement if missing:** This is **deferred to the records-management / compliance / legal functions** — request the determination before configuring. Do not invent record status, lock periods, citations, or the regulatory designation.

2.3 Roles for records management, the file plan & disposition

• **Description:** The least-privilege role groups to create and manage records in Microsoft Purview: **Records Management** (or **Compliance Administrator** / **Compliance Data Administrator**) for record labels, the file plan, and descriptors; **Disposition Management** for the named reviewer accounts; and the workload admin rights (Exchange / SharePoint) where item- or location-level application requires them.

• **Why required:** Creating record/regulatory labels, building the file plan, and acting on disposition each require the corresponding Purview role; disposition reviewers must be in **Disposition Management** or they cannot see the queue. Applying/declaring records where users do it requires the label be published to them.

• **Verify:** Microsoft Purview portal → **Roles & scopes** → **Permissions (role groups)** — confirm the admin's membership; confirm reviewers are in **Disposition Management**.

• **Implement if missing:** Have a Global Admin / Compliance Admin add the accounts to the least-privilege role groups under change control. Grant **Disposition Management** only to the named reviewer accounts, and keep records configuration separate from disposition review (duty separation).

2.4 In-scope workloads and their state (Exchange / SPO / OneDrive / Teams)

• **Description:** Confirmation of which **workloads and locations** hold the records in scope and that they are in a state a record label can apply to — Exchange mailboxes, SharePoint sites, OneDrive accounts, Microsoft 365 Groups, and Teams content — plus which of these will **publish** the record labels for manual declaration and which will **auto-apply** by condition.

• **Why required:** Record declaration keys off where the label is published/applied; a record label locking the wrong location either over-locks live content or misses the records it should cover.

• **Verify:** Confirm the in-scope workloads with stakeholders; identify the pilot locations; identify any locations that must be excluded from record declaration.

• **Implement if missing:** Define the location scope per workload before authoring labels; document any deliberate exclusions.

2.5 Existing Purview records posture (alignment, not duplication)

• **Description:** Knowledge of the organization's **existing records/retention posture** — any record or regulatory labels already in force, existing file-plan descriptors, and any retention labels/ policies the broader lifecycle program owns — so this immutability + file-plan work **aligns** to it rather than duplicating labels or conflicting via retention precedence.

• **Why required:** A duplicate record label per workload fragments the file plan; an unaware new label can interact with existing retention via Microsoft's precedence rules. The broader lifecycle/records program owns tenant-wide retention policies and must not be re-implemented here.

• **Verify:** Enumerate existing labels and descriptors (`Get-ComplianceTag`, `Get-FilePlanPropertyAuthority` and the other `Get-FilePlanProperty*` cmdlets) and confirm which are owned by the broader program (the assessment script does this).

• **Implement if missing:** Engage the lifecycle/records program owners; capture the existing baseline before adding labels or descriptors, and reuse existing descriptors rather than minting duplicates.

2.6 SharePoint records mechanics (referenced, not rebuilt)

• **Description:** For the SharePoint portion of scope, the **SharePoint-specific records mechanics** — default library labels, in-place records inside document libraries, and PnP-driven site/library application — which are implemented in depth by the **SharePoint records/retention product**.

• **Why required:** This solution configures the record/regulatory labels and the file plan; the SharePoint-library-level application detail is that product's province. Duplicating it risks divergent guidance and divergent label names.

• **Verify:** Confirm whether the SharePoint records/retention product is in play for SPO sites; if so, align label names and defer library-application mechanics to it.

• **Implement if missing:** Where SPO library-level records work is required, use the SharePoint records/retention product for that mechanic and keep the label inventory consistent across both.

2.7 Records/file-plan automation toolchain

• **Description:** The **Security & Compliance PowerShell** endpoint (`Connect-IPPSSession` from the **ExchangeOnlineManagement** module) for record labels, regulatory labels, and file-plan descriptors (`Get-ComplianceTag` / `New-ComplianceTag`, the `New-FilePlanProperty*` / `Get-FilePlanProperty*` family), and **Microsoft Graph PowerShell** for supporting directory reads — a modern-auth / MFA- capable admin.

• **Why required:** The provided scripts and the configuration steps run through Security & Compliance PowerShell (and Graph for supporting reads); both must connect, and the records/file-plan cmdlets must be present (they depend on the plan and role in 2.1/2.3).

• **Verify:** `Connect-IPPSSession` succeeds and `Get-ComplianceTag` / `Get-FilePlanPropertyAuthority` return without error; `Connect-MgGraph` succeeds where used. In Government clouds, confirm the correct `-ConnectionUri` / endpoint.

• **Implement if missing:** Install the **ExchangeOnlineManagement** and **Microsoft.Graph** modules and confirm modern-auth sign-in before running steps. Never store credentials in scripts.

2.8 Change control and a documented rollback — with the irreversible parts flagged

• **Description:** An approved change window and a **rollback plan** specific to records: how to unpublish a record label policy, how to remove a non-regulatory record label from items, and how a records manager **un-declares** a non-regulatory record — **plus the explicit statement that a regulatory-record label, once applied, cannot be removed, changed, un-declared, or shortened**.

• **Why required:** Records declaration is high-impact and partly irreversible; a documented, rehearsed rollback is the safety net for the reversible parts and forces a conscious, signed-off decision on the irreversible **regulatory** parts before they are applied.

• **Verify:** Confirm a change ticket, an approver, and a written rollback covering label-policy unpublish, non-record-label removal, and non-regulatory record un-declaration for the pilot categories — and an explicit **no-rollback** flag on every regulatory-record category.

• **Implement if missing:** Author the rollback (Section 3 provides the steps), get change-control sign-off, and explicitly flag any **regulatory-record** category as **no-rollback** with a records/ legal sign-off reference before applying it.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

Vendor boundary (applies to every Purview solution). *The* ***Microsoft Purview portal*** *(purview.microsoft.com), the* ***Microsoft 365 admin center****,* ***Security & Compliance PowerShell*** *(`Connect-IPPSSession`), and* ***Microsoft Graph*** *are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily* ***licensing-/plan-dependent*** *(many require* ***Microsoft 365 E5*** *or the* ***E5 Compliance*** *add-on; some are in preview) and the portal is* ***actively evolving*** *— capability availability and portal paths are* ***cited or flagged "verify your licensing/tenant"****, never asserted or invented.* ***Regulatory, records-retention, and legal determinations*** *(retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are* ***deferred to the organization's compliance, records, privacy, and legal functions*** *— the solutions provide the technical configuration method and evidence, not the determination. Automation is* ***PowerShell*** *(Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).
- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.
- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.
- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.
- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.
- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide

This is the ordered, pilot-first method to deliver regulatory & immutable records and a mapped file plan in Microsoft Purview. It follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — and every artifact is named per the convention in Section 1 and reused verbatim in the scripts. **Records declaration is high-impact and — for regulatory records — irreversible**: record labels are published/auto-applied to the **pilot scope only**, regulatory labels are applied only to a disposable test item in pilot and gated behind explicit sign-off, and nothing expands to production until the Section 5 gates pass. The **records determination and all legal calls are inputs from records/legal** — this guide implements them, it does not author them.

> ***Portal paths evolve and are cloud-dependent.*** *Paths below are Microsoft Purview portal (`purview.microsoft.com`) as of authoring; in GCC/GCC High/DoD the URL and some features differ (`*.office365.us`).* ***Verify the path and feature availability in your tenant/cloud*** *before relying on it. Where a step is plan-dependent, it is flagged.*

Stage 0 — Connect and confirm prerequisites

1. Confirm Section 2 prerequisites: compliance licensing for records/file plan (2.1), the records

determination from records/legal (2.2), roles incl. Disposition Management (2.3), in-scope workloads (2.4), existing posture (2.5), and change control with irreversible parts flagged (2.8).

1. Connect the toolchain from an MFA-capable admin workstation:

```
Connect-IPPSSession                # Security & Compliance PowerShell (add -ConnectionUri for Gov clou
Connect-MgGraph -Scopes "Directory.Read.All"   # supporting reads only
```

1. Run `Resolve-RecordsScope.ps1` to confirm module/connection health and surface plan/cloud hints

before touching anything. **Pass/fail:** both connections succeed, the script reports the expected cloud, and Records management (file-plan + records cmdlets) is reachable.

Stage 1 — Assess (read-only)

1. Run `Assess-RecordsFilePlanPosture.ps1` to produce `Records Posture – Tenant – Discovery`: the

existing labels (`Get-ComplianceTag`) with their **record / regulatory** flags, retention action and duration; the existing **file-plan descriptors** (`Get-FilePlanPropertyAuthority`, `-Category`, `-SubCategory`, `-Citation`, `-Department`, `-ReferenceId`); and which labels are currently **mapped** to descriptors vs unmapped.

1. Review the baseline with the lifecycle/records program owners (2.5): what already exists, which

descriptors can be reused, and which labels the broader program owns.

1. **Pass/fail (Assess gate):** a complete inventory export exists; existing record/regulatory labels and

file-plan descriptors are identified; reusable descriptors are flagged; no destructive action was taken (read-only).

Stage 2 — Design (map the determination to Purview)

Translate the records/legal determination into a concrete artifact map. Produce a design table with, for each in-scope category:

- **Category** → **label rung:** plain retention (out of scope here, defer to lifecycle product), **record** (immutable, reversible), or **regulatory record** (one-way). Only the last two are configured here.
- **Retention action + duration + trigger:** keep / keep-then-delete, the lock period, and created / last-modified / labeled / event trigger.
- **File-plan descriptors:** retention **authority**, **category / sub-category**, **department**, **citation** (+URL), **reference ID** — every record label must map to all that apply.
- **Application:** published (users declare) vs auto-apply (by condition) vs SharePoint default (deferred).
- **Disposition:** automatic vs **reviewed** (and the named reviewer group).
- **Locations:** which workloads (Exchange / SPO / OneDrive / Groups / Teams).

Name every artifact now (Section 1 table) and record the **pilot scope** `Scope – Records-Pilot – Static` (the bounded users/sites/mailboxes and the disposable test items to prove on). **Pass/fail (Design gate):** every in-scope category maps to a named record or regulatory label with action, duration, trigger, descriptors, application method, disposition, and locations; records/legal has **signed off** the mapping — especially each **regulatory-record** and its no-rollback flag.

Stage 3 — Configure (build, staged safe)

Build in this order. **Configuration scripts default to `-WhatIf`/dry-run and only write with `-Commit`; regulatory labels additionally require `-AllowRegulatory -SignOffRef`.**

3a. File-plan descriptors first (so every label maps from creation)

Create the descriptors the labels will reference via `Set-FilePlanDescriptors.ps1` (which wraps the `New-FilePlanProperty*` family with `-WhatIf` default). With `-Commit` it runs, for example:

```
New-FilePlanPropertyAuthority   -Name "Authority – Records-Schedule-2026"
New-FilePlanPropertyCategory    -Name "Contracts"
New-FilePlanPropertySubCategory -Name "Vendor-Contracts"
New-FilePlanPropertyCitation    -Name "SOX" -CitationUrl "https://contoso.example/citations/sox" `
                                -CitationJurisdiction "US"
New-FilePlanPropertyDepartment  -Name "Finance"
New-FilePlanPropertyReferenceId -Name "REC-CON-001"
```

Reuse any existing descriptors the assessment found rather than minting duplicates. *(Use the descriptor names your records determination defines — the above are placeholders.)*

3b. Record labels (immutable, reversible) with descriptors attached

Create the record labels via `Set-ImmutableRecordLabels.ps1` (which wraps `New-ComplianceTag` with `-WhatIf` default). Each carries its action, duration, trigger, `-IsRecordLabel $true`, and the **file-plan property** built from the descriptors above. The intent the script commits with `-Commit`:

```
# Record label (immutable, reversible) — date-based, mapped to the file plan
New-ComplianceTag -Name "Record Label – Contracts-7yr – Pilot" -IsRecordLabel $true `
  -RetentionAction KeepAndDelete -RetentionDuration 2555 -RetentionType ModificationAgeInDays `
  -FilePlanProperty $filePlanProperty
```

The `$filePlanProperty` object is built from the descriptor names per Microsoft's documented file-plan property format (authority / category / sub-category / citation / department / reference ID) — the script assembles it for you from the design JSON. Applying this label **declares an in-place record** and locks the item; a records manager can later **un-declare** it (evidenced) because it is non-regulatory.

3c. Regulatory-record labels (one-way) — gated, pilot only on a disposable item

Only for categories records/legal designate as irreversible, and only with the explicit switch and a sign-off reference:

```
# Regulatory record label — ONE-WAY: cannot be removed, changed, un-declared, or shortened.
New-ComplianceTag -Name "Regulatory Record Label – Financial-7yr – Pilot" -Regulatory $true `
  -RetentionAction Keep -RetentionDuration 2555 -RetentionType CreationAgeInDays `
  -FilePlanProperty $filePlanProperty
```

> ***Regulatory records are irreversible.*** `Set-ImmutableRecordLabels.ps1` *refuses to set* `-Regulatory` *without* `-AllowRegulatory -SignOffRef "<records/legal ref>"`*. In pilot, apply a regulatory label* ***only to a disposable test item you are prepared to keep for its full period*** *— never to real content.*

3d. Label policies (publish / auto-apply to the pilot)

Publish the record labels to pilot users (manual declaration) or auto-apply by condition, scoped to the pilot:

```
# Publish record labels for users to declare (pilot scope)
New-RetentionCompliancePolicy -Name "Label Policy – Records-Publish – Pilot" `
  -ExchangeLocation "pilotuser@contoso.com" `
  -SharePointLocation "https://contoso.sharepoint.com/sites/RecordsPilot"
New-RetentionComplianceRule -Policy "Label Policy – Records-Publish – Pilot" `
  -PublishComplianceTag "Record Label – Contracts-7yr – Pilot"

# Auto-apply a record label by condition (pilot scope) — condition from the determination, not invented
New-RetentionCompliancePolicy -Name "Label Policy – Records-AutoApply – Pilot" `
  -SharePointLocation "https://contoso.sharepoint.com/sites/RecordsPilot"
New-RetentionComplianceRule -Policy "Label Policy – Records-AutoApply – Pilot" `
  -ApplyComplianceTag "Record Label – Contracts-7yr – Pilot" -ContentMatchQuery '"master services agreemen
```

(SharePoint default-library-label application is deferred to the SharePoint records/retention product.)

3e. Disposition reviewers

Ensure the named reviewer accounts are in the **Disposition Management** role group (2.3) so any reviewed-disposition categories can be actioned in the portal. Keep records configuration and disposition review as separate duties. **Pass/fail (Configure gate):** every designed descriptor and label exists, every record label is **mapped** to its file-plan descriptors, regulatory labels exist only where signed off, label policies are **pilot-scoped**, reviewers are in Disposition Management, and no label is published to production yet.

Stage 4 — Pilot (prove on the bounded scope)

1. Confirm the label policies publish/auto-apply to the **pilot only**.
2. **Declare a record:** as a pilot user, apply `Record Label – Contracts-7yr – Pilot` to a disposable

test item, then attempt to edit/delete/move it — it must be **locked**.

1. **Regulatory (test only):** apply the regulatory label to a **single disposable test item you will

keep for its full period**; confirm it cannot be removed or changed. Do this only if a regulatory category is in pilot scope and signed off.

1. **Auto-apply:** create content matching an auto-apply condition; allow for **propagation** (hours to days — up to 7 days for existing content) and confirm the record label applies and locks.

1. Allow for propagation, then run the Stage 5 validation.

2. **Pass/fail (Pilot gate):** on pilot content, record labels declare and **lock** as designed, every applied label is **mapped** to the file plan, the regulatory lock is confirmed irreversible on the disposable test item, and **no content outside the pilot scope is affected**.

Stage 5 — Validate

Run `Get-RecordsFilePlanReport.ps1` to produce `Records & File Plan – Tenant – Report` and work the Section 5 test procedures. Every gate has an explicit pass/fail. Do not expand until all pass.

Stage 6 — Expand (enforce to production scope)

1. Mint the **production** artifacts by swapping `– Pilot` for `– Prod` (never edit a validated pilot artifact in place, and never re-declare a regulatory record): e.g. `Record Label – Contracts-7yr – Prod`, `Label Policy – Records-Publish – Prod`. Descriptors are reused as-is (they are not stage-suffixed).

1. Broaden the label-policy locations to the full in-scope workloads (or attach **adaptive scopes** so coverage tracks the estate), and publish/auto-apply under change control.

1. Stage the rollout in **rings** (by department/site) rather than a single flip where the determination allows; re-run reporting after each ring.

1. **Pass/fail (Expand gate):** production record labels declare and lock across the in-scope workloads, the report shows every record label mapped to the file plan, regulatory categories are applied only where signed off, and no unexpected retention conflicts appear.

Rollback (per stage)

• **Label policies (reversible):** unpublish a record label by disabling/removing its rule (`Set-RetentionComplianceRule` / remove the rule) or reducing its locations; disable an auto-apply policy so no further content is declared.

• **Record labels — non-regulatory (reversible):** a **records manager** can **un-declare** a non-regulatory record (auditable), restoring edit/delete; a published label can be unpublished and an unused non-record label deleted. **Do this only per records/legal direction.**

• **File-plan descriptors (reversible before use):** an unused descriptor can be removed; a descriptor in use by a label is retained until the label is retired.

- **IRREVERSIBLE — no rollback:** a **regulatory-record** label (cannot be removed, changed, un-declared,

or shortened) and any **completed disposition** (content permanently deleted). These are gated on records/legal sign-off and must never be exercised beyond a single disposable test item in pilot.

Every production change carries a change ticket, an approver, and this rollback written out for the specific categories in scope, with the **regulatory** categories explicitly flagged **no-rollback**.

5. Testing & Validation

Ten procedures, each with an explicit **pass/fail**. Run them at the **Pilot** and **Expand** gates. Most records behavior is **eventually consistent** — allow for service propagation (hours to a few days; up to 7 days for auto-apply of existing content) before declaring a fail. Use the provided scripts where noted; **never** apply a regulatory-record label to any item you are not prepared to keep for its full period, and never approve a real disposition as a test.

T1 — Toolchain & scope health

- **Do:** Run `Resolve-RecordsScope.ps1`.
- **Pass:** `Connect-IPPSSession` (and Graph where used) succeed; the correct cloud is reported; Records

management (file-plan + records cmdlets) is reachable; the pilot scope resolves.

- **Fail:** any connection fails, the wrong cloud/endpoint is reported, records/file-plan cmdlets are

absent (plan/role), or the scope does not resolve.

T2 — Baseline inventory captured (read-only)

- **Do:** Run `Assess-RecordsFilePlanPosture.ps1`.
- **Pass:** `Records Posture – Tenant – Discovery` lists existing labels with record/regulatory flags,

action/duration, and the existing file-plan descriptors; labels are correctly flagged as mapped vs unmapped; reusable descriptors are identified.

- **Fail:** the inventory is incomplete, mis-flags record/regulatory status, or the script wrote any

change (it must be read-only).

T3 — File-plan descriptors created as designed

- **Do:** Run the `Get-FilePlanPropertyAuthority` / `-Category` / `-SubCategory` / `-Citation` / `-Department` / `-ReferenceId` cmdlets (or `Get-RecordsFilePlanReport.ps1`) for each designed descriptor.
- **Pass:** every designed descriptor exists with the correct name (and, for citations, URL/jurisdiction),

and reused descriptors were not duplicated.

- **Fail:** a designed descriptor is missing, misnamed, or duplicated.

T4 — Record labels created and mapped to the file plan

• **Do:** `Get-ComplianceTag` for each designed record label.

• **Pass:** each record label exists with `IsRecordLabel` true, the correct action/duration/trigger, and its **file-plan descriptors attached** (authority/category/department/citation/reference ID as designed) — matching the Stage 2 design.

• **Fail:** any record label is missing, is not flagged as a record, has the wrong action/duration, or is **unmapped** (no file-plan descriptors).

T5 — Regulatory-record labels exist only where signed off

• **Do:** `Get-ComplianceTag` and filter for `IsRegulatory`/`Regulatory` true.

• **Pass:** a regulatory-record label exists **only** for categories records/legal signed off, each is mapped to the file plan, and each has a recorded sign-off reference; no regulatory label exists for a non-designated category.

• **Fail:** a regulatory label exists without sign-off, a designated regulatory category is a plain record label instead, or a regulatory label is unmapped.

T6 — Record declaration locks content

• **Do:** As a pilot user, apply a **record** label to a disposable pilot test item; attempt to edit/delete/move it.

• **Pass:** the item is declared an in-place record and is **locked** against edit/delete/move per its rules, with a declare entry in the audit trail. Un-declaration by a records manager restores edit/delete (non-regulatory).

• **Fail:** the item remains editable/deletable, or no record was declared.

T7 — Regulatory lock is irreversible (disposable test item only)

• **Do:** *Only if a regulatory category is in pilot scope and signed off:* apply the regulatory label to a **single disposable test item you will keep for its full period**; attempt to remove/change the label and un-declare it.

• **Pass:** the regulatory label cannot be removed, changed, un-declared, or shortened by any role — the lock is confirmed one-way.

• **Fail:** the regulatory label can be removed/changed/un-declared. *(Never run this on content you are not prepared to keep permanently.)*

T8 — Publish / auto-apply reaches the pilot only

• **Do:** As a pilot user, confirm published record labels are selectable (Outlook/SharePoint/OneDrive); create content matching an auto-apply condition and wait for propagation.

• **Pass:** published record labels appear for **pilot users only**; auto-apply applies and **locks** the designed record label on matching pilot content; no content outside the pilot scope is affected.

- **Fail:** labels do not appear, apply to the wrong scope, or the wrong label is auto-applied.

T9 — Every declared record maps to an authority/citation (defensibility)

- **Do:** Run `Get-RecordsFilePlanReport.ps1` and inspect the file-plan mapping columns.
- **Pass:** every in-scope record/regulatory label reports a **retention authority** and **citation** (and category/department/reference ID as designed) — the inventory is fully mapped and traceable to the mandate.
- **Fail:** any in-scope record label is unmapped (no authority/citation), i.e. locked without a documented reason.

T10 — Records & file-plan report reflects reality

- **Do:** Run `Get-RecordsFilePlanReport.ps1` to produce `Records & File Plan – Tenant – Report`.
- **Pass:** the report shows record/regulatory label counts, file-plan mapping completeness, label-policy locations/enabled state, and disposition-reviewer pointers consistent with what was configured and observed in T3–T8.
- **Fail:** the report disagrees with observed state, or omits configured labels/descriptors.

Overall gate. The pilot passes only when T1–T10 pass on the pilot scope with no effect outside it. Expansion passes only when T3–T10 pass at production scope with every record label mapped to the file plan and no unexpected retention conflicts. Any fail on an **irreversible** artifact (a regulatory record, a completed disposition) halts the rollout pending records/legal review.

6. Troubleshooting

Common failure modes when delivering regulatory & immutable records and a mapped file plan in Microsoft Purview, and how to resolve them. Most records behavior is **eventually consistent** — before treating something as broken, confirm it is not simply **propagation latency** (see the last item). Portal paths and feature availability are **cloud-dependent** — verify in your tenant/cloud.

Cannot connect / records or file-plan cmdlets missing

- **Symptom:** `Connect-IPPSSession` fails, or `Get-ComplianceTag` / `New-ComplianceTag` / `New-FilePlanPropertyAuthority` / the other `New-FilePlanProperty*` cmdlets are "not recognized."
- **Likely cause:** Not connected to Security & Compliance PowerShell; the **ExchangeOnlineManagement** module is missing/old; or the session lacks the **Records management** plan/role.
- **Resolve:** Install/update `ExchangeOnlineManagement`; run `Connect-IPPSSession` (with `-ConnectionUri`

for GCC/GCC High/DoD). Run `Resolve-RecordsScope.ps1` to confirm connectivity, cloud, and reachable cmdlets. If the records/file-plan cmdlets are absent, confirm the **E5 / E5 Compliance-class** licensing (2.1) and the **Records Management / Compliance Administrator** role (2.3).

"Insufficient permissions" creating labels or the file plan

- **Symptom:** Access-denied on `New-ComplianceTag` or `New-FilePlanProperty*`.
- **Likely cause:** The admin is not in the required Purview role group.
- **Resolve:** Add the account to **Records Management** or **Compliance Administrator** /

Compliance Data Administrator (Purview portal → Roles & scopes → Permissions). Reviewers need **Disposition Management**. Re-check with `Get-RoleGroupMember` / the portal after assignment (role propagation can take time).

`Set-ImmutableRecordLabels.ps1` *REFUSES a label as "UNMAPPED"*

- **Symptom:** The config script refuses to create a record label, reporting it is unmapped.
- **Likely cause:** The label's `filePlan` block in the design has no **authority** and/or **citation** —

by policy this solution will not create an immutable record with no documented reason for the lock.

- **Resolve:** Add the file-plan descriptors (authority, citation, and category/department/reference ID as

designed) to the label in the design JSON, run `Set-FilePlanDescriptors.ps1 -Commit` to create any missing descriptors, then re-run the label script. This is a guardrail, not a bug.

A record label isn't locking content

- **Symptom:** After applying a record label, the item can still be edited/deleted.
- **Likely cause:** The label was created without `-IsRecordLabel $true` (so it is a plain retention

label); declaration propagation hasn't completed; or a plain retention label was applied instead of the record label.

- **Resolve:** Verify with `Get-ComplianceTag` that `IsRecordLabel` is true on the applied label; confirm

the **correct** label was applied; allow for propagation. Remember a **regulatory** record is a stronger, one-way lock — don't confuse the two rungs.

Regulatory vs record confusion — a label is more (or less) locked than intended

- **Symptom:** A label can't be un-declared when you expected it to be reversible, or vice versa.
- **Likely cause:** `-Regulatory $true` (one-way, irreversible) was used where a plain `-IsRecordLabel`

(reversible, un-declarable) was intended, or the reverse.

- **Resolve:** Check `IsRegulatory` / `IsRecordLabel` with `Get-ComplianceTag`. A **regulatory** label

cannot be changed or removed — if one was created in error it cannot be un-declared; retire it from publication and create the correct label. This is exactly why the config script gates `-Regulatory` behind `-AllowRegulatory -SignOffRef` and why regulatory labels are only ever tested on a disposable item.

A published record label doesn't appear for users

- **Symptom:** Pilot users don't see the record label in Outlook/SharePoint/OneDrive to declare a record.
- **Likely cause:** Label-publish **propagation delay** (can take a day or more); the label policy's locations don't include the user; or the client hasn't refreshed.
- **Resolve:** Confirm the label policy (`Label Policy – Records-Publish – Pilot`) targets the pilot user/site; allow up to a few days; have the user sign out/in. Verify with `Get-RetentionComplianceRule` that `PublishComplianceTag` names the correct label.

Auto-apply isn't declaring matching content

- **Symptom:** Content that should match an auto-apply condition stays undeclared/unlabeled.
- **Likely cause:** Auto-apply runs on a service timer and can take **7+ days** for existing content; the `ContentMatchQuery` / sensitive-info-type condition doesn't actually match; the location isn't in scope; or the plan doesn't include auto-apply.
- **Resolve:** Confirm the condition against real content (test the KQL query), confirm the location is in the policy, confirm licensing, and **wait for the timer** before judging. Auto-apply of a record label is subject to the same latency.

The report shows an UNMAPPED record label

- **Symptom:** `Get-RecordsFilePlanReport.ps1` flags a record/regulatory label as UNMAPPED (no authority/citation).
- **Likely cause:** The label was created (perhaps in the portal, or by bulk import) without file-plan descriptors attached — a record locked without a documented reason.
- **Resolve:** Attach the file-plan descriptors: in the portal edit the label's file-plan descriptors, or for a **non-regulatory** label re-create it mapped and re-publish. A **regulatory** label cannot be edited — if it is unmapped, record the authority/citation out-of-band and factor it into the next version, and treat it as a lesson to always map before creating.

Disposition items never reach reviewers

- **Symptom:** Content expires but nothing appears in the Disposition queue, or reviewers can't see it.
- **Likely cause:** The label has no `ReviewerEmail`; reviewers aren't in **Disposition Management**; or retention hasn't actually elapsed yet. (The disposition *operation* is owned by the broader records product; this solution wires the role and reports coverage.)
- **Resolve:** Confirm the label's reviewers (`Get-ComplianceTag`), add the reviewer accounts to **Disposition Management**, and confirm the retention period has genuinely expired. Disposition is actioned in **Records management** → **Disposition** in the portal — the report points there rather than approving disposals itself.

Government-cloud portal path or feature differs / is missing

- **Symptom:** A portal path from the guide 404s, an endpoint fails, or a records/file-plan feature isn't

present in GCC/GCC High/DoD.

• **Likely cause:** Sovereign-cloud differences — different URLs (`*.office365.us`), different `Connect-IPPSSession -ConnectionUri`, and some features lag or are unavailable.

• **Resolve:** Use the correct sovereign endpoint and portal URL for your cloud, and **verify feature availability in your tenant** before relying on it. Treat any Commercial-vs-Government difference as a tenant-verify item, not an error.

It "isn't working" — but it's actually propagation latency

• **Symptom:** A just-made change (label publish, auto-apply, record declaration) hasn't taken effect.

• **Likely cause:** Normal **eventual consistency** — publish, auto-apply, and declaration run on service timers (hours to several days; some auto-apply of existing content up to 7 days).

• **Resolve:** Wait the expected window before re-testing; don't stack "fixes" on top of a change that simply hasn't propagated. Re-run `Get-RecordsFilePlanReport.ps1` after the window to confirm state.

7. Best Practices

Durable practices for delivering and running regulatory & immutable records and a mapped file plan in Microsoft Purview. They encode the house model — pilot-first, evidence-based, records/legal-owned determinations — into day-to-day operation.

Governance & ownership

• **Records/legal own the determination; you own the implementation.** Never invent record status, a lock period, a citation, or — above all — the **regulatory** designation. Treat the records determination as the source of truth and keep a traceable link from each label back to its authority and citation (the file-plan descriptors exist for exactly this).

• **Keep a single label inventory.** One record (or regulatory) label per determination category, named per the convention, reused across workloads and across the SharePoint records product — never mint duplicate labels per workload; it fragments the file plan.

• **Separate duties.** Records managers configure labels and the file plan; **Disposition Management** reviewers act on the queue and nothing else; workload admins apply where needed. Grant least privilege and review membership periodically.

Immutability discipline — record vs regulatory record

• **Reserve regulatory records for genuine legal mandates.** A regulatory-record label is **one-way** — it cannot be removed, changed, un-declared, or shortened. Use it only where records/legal explicitly require an irreversible lock, gate it behind an explicit sign-off reference, and **never** apply it to content — even in pilot — beyond a single disposable test item you are prepared to keep for its full period.

- **Prefer the reversible rung when it satisfies the mandate.** A **record** label already locks content

against edit and early deletion, and a records manager can un-declare it if the schedule allows. If the determination does not demand irreversibility, a record label is the safer default.

- **Communicate the lock before you apply it.** A record label removes edit/delete by design; tell content

owners before declaring records in a live location, and be explicit that regulatory records can never be reversed.

The file plan — map everything, from day one

- **Build descriptors before labels.** Create the authority, category/sub-category, department, citation,

and reference-ID descriptors first, then attach them as each label is created — never leave a record label unmapped and "fill it in later."

- **Every immutable record must trace to an authority and citation.** A record locked without a documented

reason is an audit finding waiting to happen. The report's mapping-completeness check exists to catch exactly this — treat any unmapped record label as a defect.

- **Reuse descriptors; don't duplicate them.** Enumerate existing descriptors during assessment and reuse

them so the file plan stays coherent across the whole records program.

- **Use bulk import/export for scale.** For large records schedules, use the portal's file-plan

import/export (CSV) to create labels-with-descriptors in bulk, then verify with the report — but keep names consistent with the convention.

Change-safety

- **Publish to pilot, then expand.** Record labels are published/auto-applied to the pilot scope; a

regulatory label touches only a disposable test item in pilot; production is a deliberate, change-controlled step after the gates pass.

- **Ring your rollout.** Expand by department/site/workload ring rather than a single flip; re-run the

report after each ring.

- **Write the rollback — and flag the irreversible parts.** Reversible: unpublish label policies, remove

non-record labels, un-declare non-regulatory records. **No-rollback:** regulatory records and completed disposition. Every change ticket states which is which, with a sign-off reference on the irreversible categories.

Operations & evidence

- **Report on a cadence.** Run the records & file-plan report regularly; watch record/regulatory counts,

file-plan mapping completeness, and label-policy coverage — an unmapped or mis-declared record is an operational risk, not just a metric.

• **Respect eventual consistency.** Publish, auto-apply, and declaration run on service timers (hours to

days; up to 7 days for auto-apply of existing content). Build that latency into expectations and never judge a change in the first minutes.

• **Point disposition to the portal.** Reviewed disposal is actioned in **Records management →

Disposition**; the report points reviewers there rather than approving disposals from a script, and the disposition *operation* is owned by the broader records product.

• **Verify by cloud.** In GCC/GCC High/DoD, confirm portal paths, endpoints (`Connect-IPPSSession

-ConnectionUri`), and feature availability — do not assume Commercial parity.

• **Keep automation credential-free.** All scripts use interactive modern auth; never embed secrets. Keep

assessment/reporting read-only and configuration `-WhatIf`-by-default with an explicit `-Commit`, and keep regulatory writes behind `-AllowRegulatory -SignOffRef`.

Alignment with the broader program

• **Don't rebuild what the lifecycle product owns.** Tenant-wide retention policies, event-based-retention

programs, and the full disposition-review operation belong to the broader data lifecycle & records product — align to them (keep label and descriptor names consistent), don't duplicate.

• **Defer SharePoint library mechanics.** Default library labels, in-place records inside libraries, and

PnP-driven application are the SharePoint records/retention product's province — keep label names consistent and defer that mechanic rather than re-documenting it.

• **Align to sensitivity labels and DLP, don't re-implement them.** Those are owned by the broader Purview

program; this solution is the immutability + file-plan layer only.

8. Deliverables Summary

Everything included in this solution, on one page.

Documentation

• **Solution Overview** — what the immutability + file-plan layer is; the **retention vs record vs

regulatory-record **distinction; how record labels lock content, how regulatory-record labels are the one-way lock, and how the** mapped file plan** (authority / category / department / citation / reference ID) makes records reportable and defensible; the reference design; what's deferred to records/legal, to the broader lifecycle & records product, and to the SharePoint records product; limitations; and the deployment model.

• **Requirements Guide** — solution-specific prerequisites (compliance licensing for records management &

the file plan; the records determination from records/legal; records/file-plan/disposition roles incl. Disposition Management; in-scope workloads; existing records posture; SharePoint mechanics referenced not rebuilt; the S&C PowerShell / Graph toolchain; change control + rollback with the irreversible parts flagged), each with how to verify and implement, building on the Purview category global requirements.

- **Implementation Guide** — Assess → Design → Configure → Pilot → Validate → Expand, with every artifact

named, file-plan descriptors built first, record labels created and **mapped**, regulatory labels gated behind explicit sign-off and applied only to a disposable test item in pilot, label policies pilot- scoped, explicit pass/fail gates per stage, and a documented rollback that flags the **irreversible** parts (regulatory records, completed disposition).

- **Testing & Validation** — 10 procedures, each with an explicit pass/fail, covering toolchain/scope,

baseline inventory, file-plan descriptors, record labels mapped, regulatory labels signed-off-only, record declaration locking, the irreversible regulatory lock (disposable test only), publish/auto-apply scope, mapping completeness (authority/citation), and the records & file-plan report.

- **Troubleshooting** — common failure modes (connection/roles/plan, record labels not locking, regulatory

vs record confusion, unmapped labels, auto-apply latency, disposition not routing, Government-cloud paths, propagation latency) and resolutions.

- **Best Practices** — governance/ownership, immutability discipline (record vs regulatory), the file plan

(map everything from day one), change-safety, operations & evidence, and alignment with the broader program.

- **Executive Summary** — the outcome, approach, and why it is low-risk, for decision-makers.

PowerShell scripts (5)

All use modern-auth `Connect-IPPSSession` (Security & Compliance PowerShell) and `Connect-MgGraph` where noted; **no hardcoded credentials**. Assessment and reporting are read-only; configuration scripts default to dry-run / `-WhatIf` and only write when `-Commit` is passed; regulatory-record actions are additionally gated behind an explicit `-AllowRegulatory -SignOffRef`.

1. `Resolve-RecordsScope.ps1` — *Scope/health helper (read-only).* Checks S&C PowerShell (and optional

Graph) connectivity, reports the detected cloud/endpoint, confirms the Records management / file-plan cmdlets are reachable, and resolves the pilot scope — the preflight before Stage 3.

1. `Assess-RecordsFilePlanPosture.ps1` — *Discovery (read-only).* Inventories existing labels

(`Get-ComplianceTag`) with **record / regulatory** flags, action, and duration, and the existing **file-plan descriptors** (`Get-FilePlanProperty*`); flags each label as **mapped vs unmapped** and identifies reusable descriptors. Produces `Records Posture – Tenant – Discovery` (CSVs).

1. `Set-FilePlanDescriptors.ps1` — *File-plan descriptor config helper (`-WhatIf` default).*

Creates/verifies the **file-plan descriptors** (`New-FilePlanPropertyAuthority` / `-Category` / `-SubCategory` / `-Citation` / `-Department` / `-ReferenceId`) from a JSON design, reusing existing descriptors rather than duplicating. Dry-run by default; writes with `-Commit`.

1. `Set-ImmutableRecordLabels.ps1` — *Record + regulatory label config helper (`-WhatIf` default).*

Creates/verifies **record labels** (`New-ComplianceTag -IsRecordLabel`) with the **file-plan property attached** from the design, and — only with `-AllowRegulatory -SignOffRef` — **regulatory-record labels** (`-Regulatory`). Dry-run by default; writes with `-Commit`; refuses regulatory labels without the explicit switch + sign-off reference, and refuses to create an **unmapped** record label.

1. `Get-RecordsFilePlanReport.ps1` — *Records & file-plan compliance report (read-only).* Compiles

record/regulatory label counts, **file-plan mapping completeness** (every record label → authority/citation/category/department/reference ID), label-policy locations + enabled state, and disposition-reviewer pointers into `Records & File Plan – Tenant – Report` (CSV + HTML), flagging any **unmapped** record label and pointing reviewers to the portal Disposition queue.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

Assess-RecordsFilePlanPosture.ps1

Get-RecordsFilePlanReport.ps1

Resolve-RecordsScope.ps1

Set-FilePlanDescriptors.ps1

Set-ImmutableRecordLabels.ps1